

Обзор методологий гибкой разработки

Методологии гибкой разработки (agile) становятся всё более популярными в индустрии программного обеспечения. В этой главе мы даём введение в гибкие методологии разработки и обзор четырёх конкретных методологий:

- Экстремальное программирование
- Методы Crystal
- Scrum
- Разработка на основе функций

Методы, основанные на плане, работают лучше всего, когда разработчики могут определить требования заранее... и когда требования относительно стабильными, с скоростью изменения порядка одного процента в месяц.

-- Барри Бём [11]

Методы, основанные на плане, начинаются с запроса и документирования набора требований, который является не только полным, но и как можно более точным. На основе этих требований затем можно сформулировать план разработки. Обычно, чем полнее требования, тем лучше план. Некоторые примеры методов, основанных на плане, - это различные каскадные подходы и другие, такие как Personal Software Process (PSP) [28] и Rational Unified Process (RUP) [30, 31]. Основополагающее предположение в процессах, основанных на плане, заключается в том, что требования относительно статичны. С другой стороны, итерационные методы, такие как подходы, основанные на спиральной модели [12, 14], эволюционные процессы, описанные в [5, 22, 32, 33], и недавно появившиеся гибкие подходы [45], рассматривают изменения и признают, что единственная константа - это изменение. Вопрос только в степени и влиянии изменения. Начиная с середины 1990-х годов специалисты-практики начали обнаруживать, что скорость изменения требований к программному обеспечению значительно превышает возможности классических методологий разработки [11, 27].

Индустрия программного обеспечения, программные технологии и ожидания клиентов менялись очень быстро, и клиенты становились всё менее склонными заранее полностью формулировать свои потребности. В результате появились гибкие методологии и практики как явная попытка более формально учитывать более частые изменения требований.

В этой главе мы представим общую информацию о принципах гибкой разработки и дадим обзор трёх гибких методологий. Для каждой из них мы представим обзор, основные роли, задействованные в методологии, создаваемые документы и артефакты, а также процесс разработки.

1 Что такое гибкость в разработке программного обеспечения?

В этом разделе мы рассмотрим модель, лежащую в основе гибкой разработки программного обеспечения.

1.1 Гибкая модель

Гибкие методы являются подмножеством итеративных и эволюционных методов [32, 33] и основаны на итеративном улучшении [5] и процессах оппортунистического развития [18].

В итеративных продуктах каждая итерация представляет собой самостоятельный мини-проект, охватывающий анализ требований, проектирование, реализацию и тестирование [32]. Каждая итерация приводит к выпуску итерации (возможно, только внутренней), которая объединяет все программные обеспечения команды и представляет собой растущее и развивающееся подмножество конечной системы. Цель коротких итераций заключается в том, чтобы обратная связь с итерацией N и более ранних, а также любая другая новая информация могли привести к уточнению адаптации требований для итерации N + 1. Заказчик адаптивно определяет свои требования к следующей версии, основываясь на наблюдении за развитием продукта, а не на догадках в начале проекта [12]. Существуют количественные доказательства того, что частые сроки снижения дисперсии процесса разработки программного обеспечения и, таким образом, могут повысить его предсказуемость и эффективность.[40]

Заранее определённая продолжительность итерации суживает временными рамками для команды. Объем работ для каждой итерации выбирается таким образом, чтобы заполнить её продолжительность. Вместо того, чтобы увеличивать продолжительность итерации для соответствия выбранному объему работ, объем работ сокращается. Ключевое отличие гибких методов от итеративных методов прошлого заключается в продолжительности каждой итерации. В прошлом итерации могли длиться от трёх до шести месяцев. В гибких методах продолжительность итераций варьируется от одной до четырёх недель и намеренно не превышает 30 дней. Исследования показали, что более короткие итерации имеют меньшую ложность и риск, лучшую обратную связь и более высокую производительность и показатели успеха [32].

Общим для всех гибких методов является признание разработки программного обеспечения эмпирическим процессом. В инженерии процессы классифицируются как определенные (также известные как предписывающие) или эмпирические (или основанные на исследовании) [36]. Определенный процесс - это тот, который можно запустить и позволить ему работать до завершения, получая результаты с небольшими вариациями каждый раз [41]. Сборка автомобиля является таким процессом. Инженеры могут разработать однозначный процесс сборки автомобиля и указать порядок сборки и действия с стороны рабочих с борочной линии, машин и роботов. Вообще говоря, если производственный процесс следует этим предопределенным шагам, можно произвести высококачественный автомобиль при условии, что программный процесс находится под контролем и допускаются подходы, т.е. дисперсия процесса низкая. Такие определенные производственные процессы подходят для создания изделий с низкой степенью новизны или изменений и с высокой повторяемостью процесса [32].

Разработка программного обеспечения часто претерпевает слишком много изменений в то время, пока команда разрабатывает продукт, чтобы считаться определенным процессом. Набор предопределенных шагов может не привести к желаемому, предсказуемому результату, поскольку разработка программного обеспечения — это, безусловно, человеческая деятельность: меняются требования, меняются технологии, люди добавляются и уходят из команды и так далее. Другими словами, дисперсия процесса высока. В инженерном контексте эмпирические процессы используются для ориентированных на исследование, часто меняющихся, возможно, нестабильных и интеллектуально-интенсивных областей [32], которые требуют постоянного мониторинга и исследовательской работы [18]. Эти условия требуют коротких циклов «инспекции и адаптации» и частых, коротких циклов обратной связи [22, 36]. Короткие циклы «инспекции и адаптации», характерные для гибких методологий, могут помочь командам разработчиков лучше справляться с противоречивыми и непредсказуемыми требованиями некоторых проектов.

Еще одной общей чертой всех гибких методологий является важность людей, исполняющих роли, и признание того, что важнее гибкое процесс или инструмента

Эти люди — самый влиятельный фактор в любом проекте. Брукс признаёт то же самое в своей книге «Мифический человек-месяц» [15]: «Качество людей, работающих в проекте, их организация и управление — более важные факторы успеха, чем используемые ими инструменты или применяемые ими технические подходы». К сожалению у некоторых гибких методов есть общие черты, которые могут быть не совсем положительными. Одна из них заключается в том, что, в отличие от более классических итеративных методов, явные количественные измерения качества, моделирование процессов и метрики часто замалчиваются, а иногда и вовсе избегаются. Возможные оправдания такого отсутствия моделирования и метрик варьируются от нехватки времени и навыков до неважности и социальных причин.

Ещё одной потенциальной проблемной областью гибких методов является способность справляться с изменениями или недосказками, внесенными в продукт. В идеале, даже в «классических» средах разработки реакция на информацию об изменении должна быть быстрой; изменения применяются в той же фазе жизненного цикла, на которой информация собирается. Однако внедрение циклов обратной связи в процесс разработки программного обеспечения будет зависеть от возможностей организации в области разработки программного обеспечения, а задержка реакции — от точности моделей обратной связи. Например, маловероятно, что организации, находящиеся ниже третьего уровня зрелости по шкале Модели зрелости возможностей (CMM) Института программного инжиниринга (SEI) [38], будут иметь процессы, способные реагировать на информацию об обратной связи менее чем за один цикл выпуска программного обеспечения. Это необходимо учитывать при рассмотрении уровня экономики «гибких» методов. Например, только относительно небольшие команды могут самоорганизоваться (один из принципов гибкой разработки) в нечто напоминающее производительность CMM уровня 4 или 5. Кроме того, поскольку кадровые ресурсы ограничены, часть программного обеспечения может оставаться непроверенной или пройти верификацию меньшей степени. Основным и самым сложным аспектом верификации системы — решить, что именно необходимо тестировать, а что можно оставлять непроверенным или протестировать частично [43].

1.2 Agile Development and Principles B

В феврале 2001 года несколько консультантов по программной инженерии объединили усилия и начали классифицировать ряд схожих чувствительных к изменениям методологий как agile (термин, который уже десятилетиями используется в гибких производственных практиках [34] и начал использоваться для разработки программного обеспечения в конце 1990-х годов [3]). Этот термин пропагандировался в первую очередь к быстрому и гибкому реагированию на изменения методологий. Консультанты сформировали Agile Alliance и написали «Манифест гибкой разработки программного обеспечения» и «Принципы, лежащие в основе Agile Manifesto» [9, 25]. Методологии, изначально принятые Agile Alliance, были адаптивная разработка программного обеспечения (ASD) [26], Crystal [17], метод разработки динамических систем (DSDM) [42], экстремальное программирование (XP) [6], разработка на основе функций (FDD) [16, 37] и Scrum [41].

1.2.1 Гибкая разработка программного обеспечения

Agile Alliance сформулировал свое заявление о ценностях [9] следующим образом:

Мы открываем лучшие способы разработки программного обеспечения, занимаясь этим и помогая другим. Благодаря этой работе мы пришли к пониманию того, что:

Индивидуумы и взаимодействия важнее процессов и инструментов

Рабочее программное обеспечение	более исчерпывающей документации по контрактным
Сотрудничество с клиентами	переговорам
Реагирование на изменения	по следованию плану

То есть, хотя предметы с права интеллектуальной собственности, мы больше ценим предметы слева.

Подразумевается, что формализация процесса разработки программного обеспечения препятствует человеческому и практическому компоненту разработки программного обеспечения, таким образом, снижает шансы на успех.

Хотя это справедливо в случае неправильного использования и понимания формализации, необходимо быть очень осторожным, чтобы не переопределять и не недоопределять элементы в левой части, поскольку это может привести к той же проблеме — низкому качеству программного обеспечения. Ключ к успеху — правильный баланс [13].

1.2.2 Принципы Agile Alliance

также задокументировал принципы, которым он следует и которые лежат в основе его манифеста [9]. Таким образом, гибкие методы основаны на принципах, а не на правилах [32].

Вместо того чтобы иметь заранее определенные правила относительно ролей, взаимоотношений и видов деятельности, команда и менеджер руководствуются следующими принципами:

1. Наш главный приоритет — удовлетворение клиента посредством своевременного и постоянного поставщика программного обеспечения.
2. Приветствуйте меняющиеся требования, даже на поздних этапах разработки. Гибкие процессы используют изменения для создания конкурентного преимущества для клиента.
3. Поставляйте работающее программное обеспечение часто, от пары недель до пары месяцев, отдавая предпочтение более коротким срокам.
4. Предприниматели и разработчики должны ежедневно работать вместе в рамках проекта.
5. Создавайте проекты, ориентируясь на мотивированных людей. Обеспечьте им необходимую среду и поддержку и доверьте им выполнение работы.
6. Самый эффективный и действенный метод передачи информации в пределах организации — это личное общение.
7. Работающее программное обеспечение является основным показателем прогресса.
8. Гибкие процессы способствуют устойчивому развитию.
9. Спонсоры, разработчики и пользователи должны иметь возможность поддерживать постоянный темп на неопределенный срок.
10. Постоянное внимание к техническому совершенству и хорошему дизайну повышает гибкость.
11. Простота — искусство максимально сократить объем невыполненной работы — имеет решающее значение.
12. Лучшие архитектуры, требования и проекты возникают в результате самоорганизации команды.
13. Команда регулярно размышляет о том, как стать более эффективной, а затем соответствующим образом траивает и корректирует свое поведение.

2 примера гибких методологий разработки программного обеспечения

В этом разделе представлено краткое введение в три гибкие методологии. Эти три были выбраны для демонстрации области применения и спецификации гибких методологий. Для каждой методологии мы приводим обзор, а затем обсуждаем

документы и артефакты, созданные командой разработчиков, роли, которые принимают на себя члены команды разработчиков, и процесс.

2.1 Экстремальное программирование (XP)

Создатели экстремального программирования (XP) [6] стремились разработать методологию подхода к созданию «объектно-ориентированных проектов, использующих команды из дюжины или меньше программистов в одном месте». [29]

Методология основана на пяти основных ценностях: общение, простота, обратная связь, смелость и уважение.

о Коммуникация. В XP существует культура усного общения, и её практики направлены на поощрение взаимодействия. Ценность коммуникации основана на наблюдении, что большинство трудностей в проекте возникает из-за того, что кто-то должен поговорить с кем-то еще, чтобы прояснить вопрос, поспорить или получить помощь. «Проблемы с проектами неизменно связаны с тем, что кто-то не обсудил с кем-то что-то важное». [6] о Простота. Разрабатывайте максимально простой продукт, отвечающий потребности клиента. Важным аспектом ценности является проектирование и кодирование только того, что указано в текущих требованиях, а не прогнозирование и планирование не установленных требований. о Обратная связь. Команда разработчиков получает обратную связь от клиентов в конце каждой итерации и внешнего релиза. Эта обратная связь определяет следующую итерацию.

Кроме того, в методологии выстроены очень короткие циклы обратной связи по проектированию и реализации, основанные на парном программировании и разработке через тестирование [44]. о Смелость. Остальные три ценности позволяют команде проявлять смелость в действиях и принятии решений. Например, команда разработчиков может обладать смелостью противостоять давлению вынуждающему брать на себя нереалистичные обязательства. о Уважение. Члены команды должны заботиться друг о друге и о проекте.

2.1.1 Документы и артефакты. В целом,

XP опирается на «документирование» посредством усного общения, с помощью кода и передачи неявных знаний, а не на письменные документы и артефакты. Однако, хотя усное общение может быть эффективным для небольших групп, оно не рекомендуется для крупных систем, систем с высоким уровнем риска или систем, требующих возможности аудита по юридическим или инженерным причинам, связанным с надежностью программного обеспечения. В этих случаях может потребоваться более формальное управление следующими «инструментами», их запись/сохранение и регулярный пересмотр в рамках более «формализованного» и простого процесса XP.

о Карточки пользовательских историй – это бумажные карточки с краткими описаниями требований (функций, исправлений, нефункциональных элементов). Карточки пользовательских историй намеренно не являются полным описанием требований, а представляют собой обязательства для дальнейшего обсуждения между разработчиком и заказчиком. В ходе этого обсуждения стороны усно соглашались, что необходимо для выполнения требования. В карточку добавляются приоритет заказчика и оценка ресурсов разработчика. Оценка ресурсов для пользовательской истории не должна превышать длительность итерации.

о Список задач, перечень задач (продолжительность от половины до трех дней) для пользовательских историй, которые должны быть выполнены за итерацию. Задачи представляют собой конкретные аспекты пользовательской истории. Программисты добровольно выполняют задачи, а не назначаются на них. о Карточки CRC [10] (необязательно), бумажная карточка, на которой записываются обязанности и участники классов, которые могут служить основой для проектирования программного обеспечения. Классы, обязанности и участники определяются в ходе мозгового штурма/ролевой игры по проектированию участия нескольких разработчиков. CRC означает Класс - Ответственность -

Сотрудничество. о Тесты приемки заказчиком, текстовые описания и автоматизированные тестовые случаи, которые разрабатываются заказчиком. Команда разработчиков демонстрирует завершение пользовательской истории и проверку требований заказчика, проходя эти тесты случаи.

о Наглядные настенные графики. Для улучшения коммуникации и подотчетности графики прогноза обычно размещаются в рабочей зоне команды. Эти графики часто показывают, сколько историй выполнено и/или сколько приемочных тестов пройдено.

2.1.2 Роли

о Менеджер, владеет командой и ее проблемами. Он или она формирует команду, получает ресурсы, управляет людьми и проблемами, а также взаимодействует с внешними группами. о Тренер, обучает членов команды процессу XP по мере необходимости, вмешивается в случае возникновения проблем; контролирует, соблюдается ли процесс XP. Тренер, как правило, программист, а не менеджер. о Трекер, регулярно собирает от разработчиков информацию о выполнении пользовательских историй и приемочных тестовых случаев для создания визуальных настенных графиков. Трекер — программист, а не менеджер или заказчик. о

Программист, пишет тесты, проектирует и кодирует; проводит рефакторинг; определяет и оценивает задачи и истории (этот человек также может быть

тестировщиком) о Тестировщик, помогает клиентам писать и разрабатывать тесты (этот человек также может быть

программистом) о Заказчик, пишет истории и приемочные тесты; выбирает истории для релиза и для итерации. Распространенное заблуждение состоит в том, что роль заказчика должен играть один человек из организации заказчика. И наоборот, можно привлечь группу клиентов или выбрать представителя клиента из числа членов организации-разработчика (но не входящего в команду разработчиков).

2.1.3 Процесс

Первоначальная версия методологии программного обеспечения XP [6], опубликованная в 2000 году, содержала 12 технических практик, ориентированных на программиста. Эти практики взаимодействуют, уравновешивают и усиливают друг друга [6, 27]. Однако в ходе опроса [20] руководителей проектов, генеральных директоров, разработчиков и вице-президентов по инжинирингу 21 программного проекта было обнаружено, что ни одна из компаний не приняла XP в «чистом» виде, когда все 12 практик использовались без адаптации. В 2005 году XP был изменен, чтобы включить 13 основных практик и 11 дополнительных практик [8]. Основные практики предназначены [8] для использования независимо друг от друга и других используемых практик, хотя взаимодействие

Сочетание практик может усилить их эффект. Дополнительные практики, вероятно, будут сложными без предварительного освоения базового набора основных практик.

Ниже кратко описаны 13 основных технических практик XP: о Сидите вместе, вся

команда разрабатывает в одном открытом пространстве. о Вся команда,
используйте кросс-функциональную команду всех тех, кто необходим для продукта.

чтобы добиться успеха

о Информативное рабочее пространство, разместите видимые настенные графики вокруг рабочего пространства, чтобы члены команды (или другие заинтересованные наблюдатели) могли получить общее представление о

том, как продвигается проект. о Энергичная работа, команды XP не работают чрезмерно с верной скоростью в течение длительных периодов времени. Мотивация этой практики — сокращение высокого качества кода (установленные программисты вносят больше дефектов) и удовлетворенность программистов (чтобы снизить текучесть кадров). Том ДеМарко утверждает, что «Увеличенная с верной скоростью работа — это метод, сдерживающий

производительность». [19] о Парное программирование [46] относится к практике, при которой два программиста работают вместе за одним компьютером, совместно работая над одним и тем же дизайном, алгоритмом, кодом или тестом. о Истории, команда пишет краткие описания видимой клиенту функциональности, которая должна быть в продукте. Разработчики оценивают историю клиент определяет приоритет истории. о Еженедельный цикл: в начале каждой недели проводится собрание для обзора прогресса и текущих задач, выбора клиентом еженедельных историй для реализации на этой неделе (на основе оценок разработчика и его собственных приоритетов), а также для разбиения историй на задачи, которые должны быть выполнены на этой неделе. К концу недели приемочные тестовые случаи для выбранных историй должны быть запущены для демонстрации клиенту, чтобы запустить следующий

еженедельный цикл. о Ежеквартальный цикл: вся команда должна выбрать тему или темы историй для квартальных историй. Темы помогают команде обдумать общую картину. В конце квартала представлять эту бизнес-ценность. о Slack: в каждой итерации планируйте несколько менее приоритетных задач, которые можно пропустить, если команда отстаёт, чтобы клиент все равно получил самую важную функциональность. о Десятиминутная сессия: структурируйте проект и

связанные с ним тесты таким образом, чтобы можно было обратиться к теме и запустить все тесты за десять минут, чтобы система была обратимой, а тесты запускались часом. о Программирование «с начала тестов»: все истории имеют как минимум один приемочный

тест, желательно автоматизированный. Если все приемочные тесты для пользовательской истории пройдены, история считается выполненной. Кроме того, автоматизированные модульные тесты пишутся инкрементально, используя практику разработки через тестирование (TDD) [7], при которой код и автоматизированные модульные тесты пишутся попеременно и инкрементально, по минуте.

о Непрерывная интеграция: программисты несколько раз в день вносят в кодовую базу готовый код и связанные с ним тесты. Код может быть внесен только в том случае, если все связанные с ним модульные тесты и все модульные тесты всей кодовой базы пройдены. о

Инкрементальное проектирование: вместе с разработкой детального проекта до внедрения, ежедневное инвестирование в проектирование с учетом прошлого опыта. Жизненный цикл и целесообразность предварительных проектов подтверждены.

В нашей нестабильной бизнес-среде произошли радикальные изменения [27]. Рефакторинг [24] для улучшения ранее написанного кода крайне важен. Команды, использующие надежные модульные тесты, могут безопасно экспериментировать с рефакторингом, поскольку у них есть подстраховка.

Ниже кратко описаны 11 основных технических практик XP:

- о Реальное участие клиента: клиент готов прояснить вопросы, связанные с требованиями, являясь экспертом в данной области и имеет право принимать решения относительно требований и их приоритетности. Кроме того, клиент пишет акт о принятии тестов.
- о Инкрементальное развертывание: постепенное развертывание функциональности в рабочей среде для снижения риска масштабного развертывания.
- о Преемственность команды: сохрание эффективной работы команд.
- о Сокращение команды: по мере роста ее мощности (благодаря опыту) поддерживайте постоянную рабочую нагрузку, но постепенно уменьшайте размер команды.
- о Анализ первопричин: изучите причину обнаруженного дефекта, написав приемочные тесты и модульные тесты для их выявления. Всплывшие выявите, почему дефекты были созданы, но не обнаружены в процессе разработки.
- о Общий код: после того, как код с данными тестами внесен в кодовую базу, код может быть изменен любым членом команды. Такое коллективное владение кодом дает каждому члену команды ощущение владения всей кодовой базой и предотвращает узкие места, которые могли бы возникнуть, если бы «владелец» компонента был недоступен для внесения необходимых изменений.
- о Код и тесты: сохраняйте только код и тесты как постоянные артефакты.
- Используйте социальные механизмы для сохранения важной истории проекта.
- о Ежедневное развертывание: запускайте новый код в эксплуатацию каждую ночь.
- о Договор о согласованном объеме работ, фиксирует время, стоимость и требуемое качество проекта, не требует постоянных переговоров об объеме работ по проекту.
- о Оплата по факту использования, взимание платы с пользователя каждый раз, когда система используется, для получения отзывов по характеру использования.

Хотя это не является «официальной» практикой XP, практически все команды XP ежедневно проводят короткие стендап-встречи [4]. На этих встречах команда встает в круг (вставание намеренно мотивирует команду к краткосрочным встречам). По очереди каждый член команды рассказывает группе: о Чего он/она достиг / добился /

добилась в предыдущий день о Что он/она планирует
сделать сегодня о Любые препятствия
или трудности, с которыми он/она столкнулся / столкнулась

Часто пары парное программирование формируются динамически во время ежедневных совещаний, когда обсуждаются задачи на день, и два программиста, которые лучше всего подходят для выполнения задачи, объединяются.

«Мужественный» XP-менеджер будет вести записи таких встреч, чтобы использовать их для количественной оценки прогресса проекта [21, 35, 43]. Бремя этой количественной оценки можно облегчить и автоматизировать с помощью соответствующих инструментов.

2.2 Кристалл

Rational Unified Process (RUP) [30, 31] — это настраиваемая структура процессов. В зависимости от характеристик проекта, таких как размер команды и размер проекта, RUP может быть адаптирован или расширен в соответствии с потребностями принимающей организации. Аналогичным образом, семейство методов Crystal [17] было разработано для учета изменчивости среды и специфических характеристик проекта. Однако RUP обычно начинается с базовой методологии, основанной на плане, и адаптируется для небольших, менее критичных проектов. Напротив, автор Crystal Алистер Кокберн считает, что базовой методологии должно быть «едва достаточно». Он утверждает: «Вам нужно на одну ступень меньше контроля, чем вы ожидаете, и чем меньше, тем лучше, когда дело доходит до быстрой поставки». [27] Более того, поскольку проект и люди с временем развиваются, методология также должна настраиваться и развиваться в ходе проекта.

Crystal – это семейство методов, поскольку Кокберн считает, что не существует универсального процесса разработки. Соответственно, различным методам присвоены цвета в порядке возрастания прозрачности; наиболее гибкая версия – Crystal Clear, за ней следуют Crystal Yellow, Crystal Orange и Crystal Red. График на рисунке 2 используется для выбора отправной точки метода Crystal (для последующей адаптации). По оси X отложен размер команды. По мере увеличения команды (сдвижение вправо по оси X) тем сложнее управлять процессом посредством личного общения и, следовательно, тем выше потребность в координации документации, практик и инструментов. Ось Y отражает потенциальный ущерб с системы. Наименьшее влияние ущерба – это потеря комфорта, затем потеря дискреционных средств, потеря средств первой необходимости и, наконец, гибель людей. Соответствующая методология Crystal определяется на основе размера команды и критичности. Каждая методология включает набор рекомендуемых практик, основной набор ролей, рабочих продуктов, методов и обозначений.

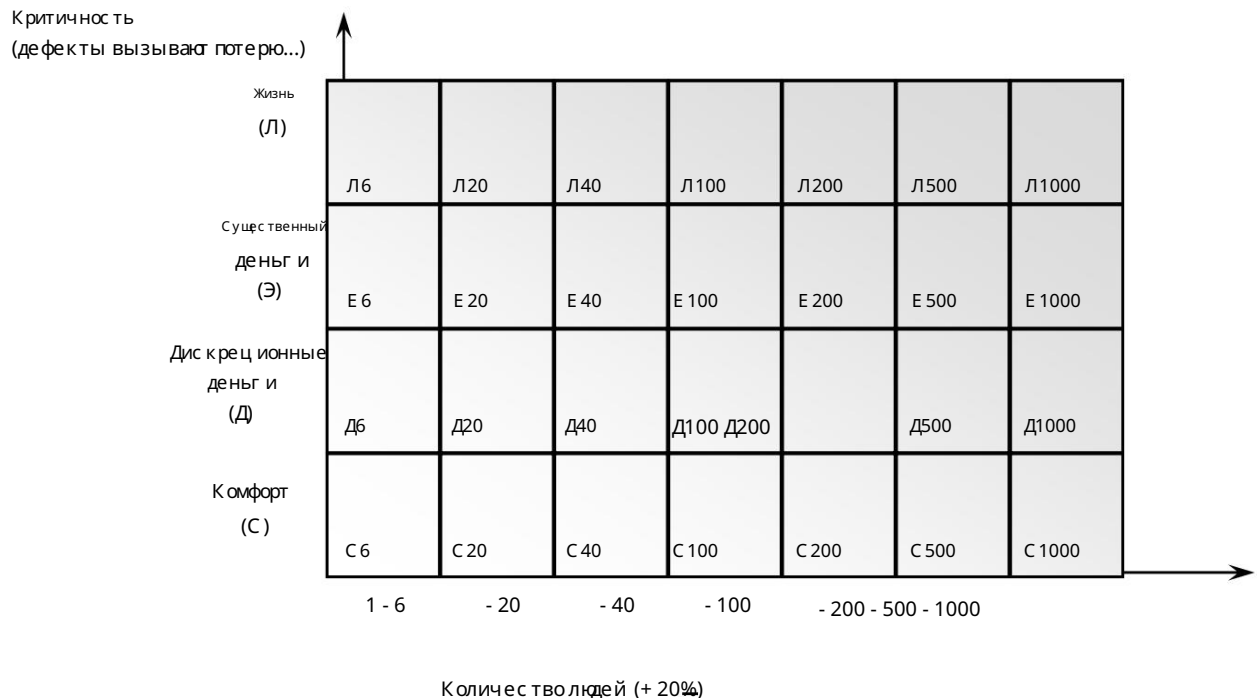


Рисунок 1: Семейство методов кристаллизации [адаптировано из [17]]

Все методы Crystal подчеркивают важность людей в разработке программного обеспечения.

«[Crystal] фокусируется на людях, взаимодействии, сообществе, навыках, талантах и коммуникации как на первостепенных факторах, влияющих на производительность. Процесс остаётся важным, но вторичным». [27]

В семействе методологий Crystal есть только два абсолютных правила. Во-первых, инкрементальные циклы не должны превышать четырёх месяцев. Во-вторых, после каждого внедрения должны проводиться семинары-рефлексии, чтобы методология могла адаптироваться. В настоящее время определены только Crystal Clear и Crystal Orange. Краткое описание этих двух методологий представлено ниже.

2.2.1 Crystal Clear. Crystal

Clear [17] ориентирован на проект уровня D6 и может быть применен к проектам уровня C6 или E6, а также, возможно, к проекту уровня D10. Crystal Clear — это оптимизация Crystal, которая может применяться, когда команда состоит из трёх-восемью человек, работающих в одной комнате или с соседних офисов. Свойственное общение упрощается до «осмотического» общения, что означает, что люди ежедневно слышат друг друга, обсуждая приоритеты, статус, требования и дизайн проекта. Элементы модели Crystal Clear следующие:

- Документы и артефакты: план выпуска, график проверок, неформальные/неформальные варианты использования, эскизы дизайна, работающий код, общая объектная модель, тестовые случаи и руководство пользователя
- Роли: спонсор/заказчик проекта, старший дизайнер-программист, дизайнер-программист и пользователь (как минимум неполный рабочий день)
- Процесс: поэтапная поставка, выпуски менее чем за два-три месяца, частично автоматизированное тестирование, прямое участие пользователей, два пользовательских отзыва на каждый релиз и ретроспективы корректировки методологии. Процесс останавливается по поставке программного обеспечения или принятым важным решениям, а не по завершённым документам.

2.2.2 Crystal Orange. Crystal

Orange предназначен для проекта D40. Crystal Orange предназначен для 20–40 программистов, работающих вместе в одном здании над проектом, в котором дефекты могут привести к потере дискретных средств (т.е. с средним уровнем риска). Продолжительность проекта составляет от одного до двух лет, и время выхода на рынок имеет важное значение. Элементы модели Crystal Clear: Документы и артефакты:

- документ с требованиями, план выпуска, описание, отчеты о состоянии, документ по дизайну пользовательского интерфейса, групповые спецификации, работающий код, общая объектная модель, тестовые случаи, код миграции и руководство пользователя
- Роли: спонсор проекта, бизнес-эксперт, эксперт по использованию технических координатор, бизнес-аналитик, менеджер проекта, архитектор, наставник по дизайну, ведущий дизайнер-программист, дизайнер-программист, дизайнер пользовательского интерфейса, точка повторного использования, писатель и тестировщик
- Процесс: поэтапная поставка, выпуски менее чем за три-четыре месяца, некоторое автоматизированное тестирование, прямое участие пользователей, два пользовательских отзыва на каждый выпуск и ретроспективы корректировки методологии.

2.3 Скрам

2.3.1 Обзор В

процесс Scrum [27, 41] управление проектами накладывается на методологию разработки программного обеспечения в виде «обертки». Методология гибка в отношении количества/малости церемоний, но философия Scrum призывает команду к максимально возможному их сокращению. Обычно Scrum-команды работают в одном месте. Однако существуют Scrum-команды, работающие географически распределённо, и участники команды участвуют в ежедневных совещаниях по громкой связи. Scrum-команды — это самоуправляемые и самоорганизующиеся команды. Команда обязуется достичь определённой цели на итерацию и получает полномочия, автономию и ответственность решать, как лучше всего её достичь.

2.3.2 Документы и артефакты

Scrum-команды создают три основных артефакта: бэклог продукта, бэклог спринта и диаграмму сгорания спринта. Все они находятся в открытом доступе и намеренно видны Scrum-команде.

- о Журнал невыполненных работ по продукту (Product Backlog) – постоянно обновляемая, приоритизированная очередь бизнес- и технических функций, которые необходимо реализовать в системе, а также дефектов, которые необходимо исправить [41] во время выпуска. Для каждого требования журнал невыполненных работ по продукту содержит уникальный идентификатор, категорию (функция, улучшение, дефект), статус, приоритет и оценку стоимости функции. Он хранится в формате электронной таблицы.
- о Бэклог спринта – список всех бизнес- и технических функций, улучшений и дефектов, запланированных для текущей итерации (называемой спринтом). Бэклог спринта также ведётся в формате электронной таблицы. Требования разбиты на задачи. Для каждой задачи в бэклоге таблицы содержится краткое описание задачи, информация об инициаторе, ответственном за задачу, статусе и количестве часов, оставшихся для её выполнения. Бэклог спринта обновляется ежедневно с помощью того, который ежедневно посещает члены команды для получения актуальных оценок оставшегося объема работы. Оценки могут быть увеличены, если член команды понимает, что объем работы был недооценен.
- о Диаграмма выработки задач спринта. Оставшиеся часы для выполнения задач спринта отображаются в графическом виде и преимущественно для команды. Пример диаграммы выработки задач представлен на рисунке 3.

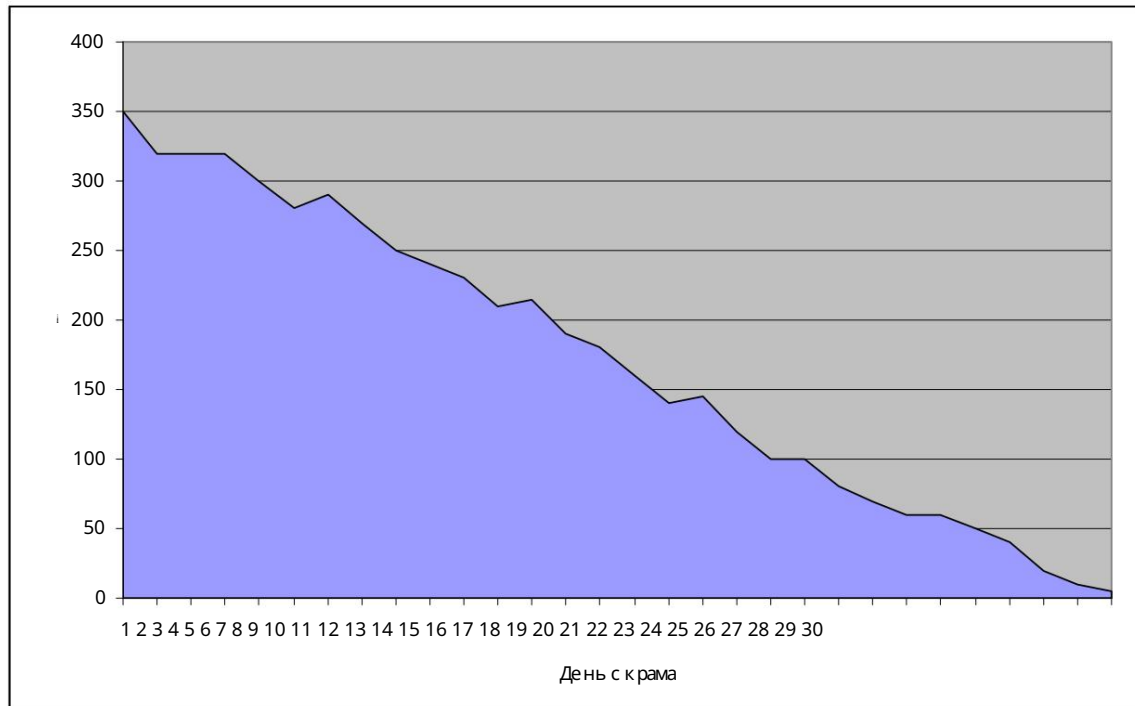


Рисунок 3: Диаграмма сгорания задан с принта

2.3.3 Роли

о Владелец продукта — человек, который отвечает за создание и приоритизацию backlog а продукта, выбирает, что будет включено в следующую итерацию с принта, и проверяет систему (совместно с другими заинтересованными сторонами) в конце с принта. о Скрам-мастер — знает и поддерживает итерационные цели продукта, а также ценности и практики Scrum, проводит ежедневные совещания (Scrum Meeting) и демонстрацию итерации (Sprint Review), отслеживает прогресс, устраняет препятствия (блоки) и предоставляет ресурсы. Скрам-мастер также является разработчиком (с.м. ниже) и участвует в разработке продукта (а не только в управлении). о Разработчик — член Scrum-команды. Scrum-команда стремится к достижению цели с принта и имеет все полномочия делать все возможное для ее достижения. Размер Scrum-команды — семь человек, плюс-минус два.

2.3.4 Процесс

Обзор процесса Scrum представлен на рисунке 4. В этом разделе будет рассмотрен каждый из элементов процесса.

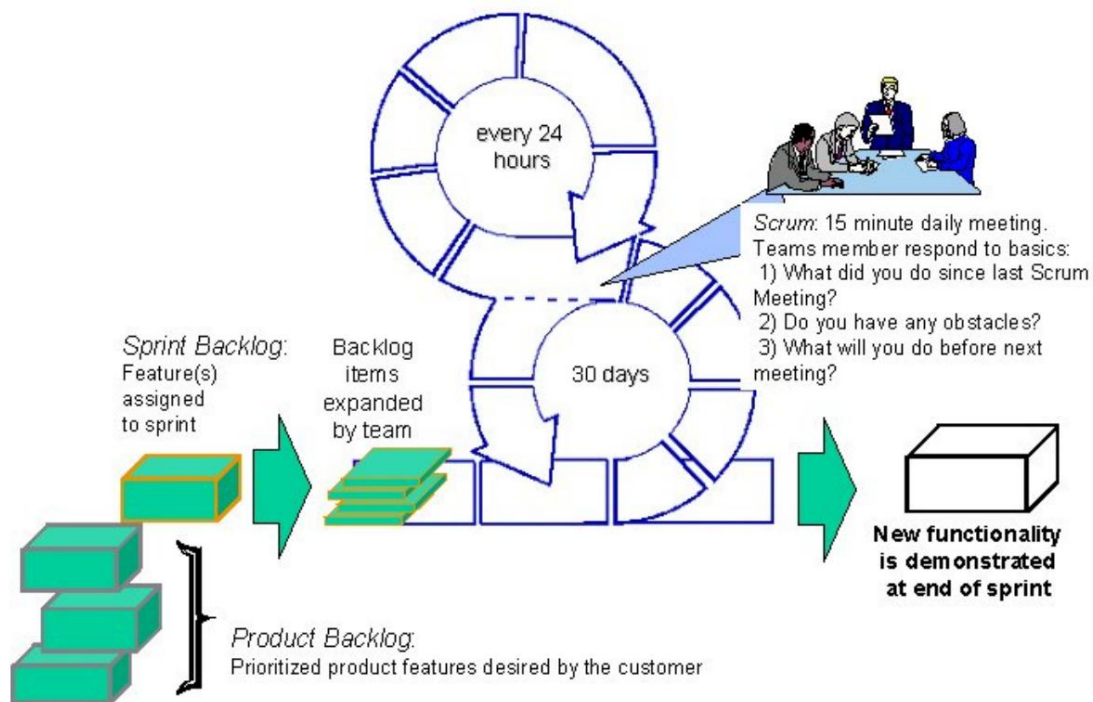


Рисунок 4: Процесс Scrum (из [1])

Процесс Scrum состоит из следующих этапов:

Проводится совещание по планированию спринта с командой разработки, руководством и владельцем продукта. Владелец продукта является представителем заказчика или группы заказчиков. Владелец продукта создает и расставляет приоритеты в бэклоге продукта. На совещании по планированию Владелец продукта выбирает, какие функции будут включены в следующий 30-дневный инкремент (называемый спринтом), обычно исходя из максимальной ценности для бизнеса и риска. Кроме того, устанавливается цель спринта, которая служит минимальным, высокоуровневым критерием успеха для спринта и позволяет команде Scrum сосредоточиться на общей картине, а не только на выбранных функциях. Команда разработки определяет задачи и ресурсы, необходимые для поставки этих функций. Совместно они определяют разумное количество функций для включения в следующий спринт. После того, как этот набор функций определен, в течение следующего 30-дневного спринта, в котором функции проектируются, внедряются и тестируются, переприоритизация не

производится. Во время спринта интегрируются и

регрессионное тестирование проводится ежедневно. Короткие 15-минутные Scrum-встречи проводятся ежедневно. Эти встречи похожи на описанные выше стендапы XP, поскольку встречи XP основаны на успехе Scrum, достигнутом с помощью Scrum-встреч. Встреча проводится в комнате с доской, на которой можно записывать задачи и блоки. Во время как другие участники (например, менеджеры) могут присутствовать на Sprint Meeting, выступать могут только участники команды и Scrum-мастер. Каждый

участник команды отвечает на следующие

вопросы: о Что вы сделали с момента последнего Scrum-

встречи? о Что вы будете делать до следующего Scrum-встречи? о Что помешало вам выполнять работу?

Scrum Meeting является важным компонентом методологии. На встрече даются социальные обещания, что, по-видимому, повышает ответственность и последовательность, а также позволяет проекту не сбиваться с пути. Однако эти встречи могут стать неуправляемыми, если в них участвует слишком много людей; рекомендуется, чтобы в каждой команде было не более семи участников. Для использования в больших командах команда подразделяется на меньшие группы, каждая из которых проводит свою собственную Scrum-встречу. Один представитель от каждой из малых групп посещает встречу «Scrum of Scrums». Этот представитель отвечает на вопросы Scrum, относящиеся к своей подкоманде. Таким образом, между подкомандами передается важная информация. В конце спринта проводится Sprint Review для обзора прогресса.

Демонстрация функций заказчику, руководству, пользователям и владельцу продукта, а также обзор проекта с технической точки зрения. Встречу проводит Scrum-мастер. Владелец продукта и другие заинтересованные стороны посещают встречу. Демонстрируется последняя версия продукта, в которой владелец продукта общается с его функциями, дизайном, сильными и слабыми сторонами, а также проблемными местами.

Основное внимание уделяется демонстрации возможностей продукта; формальные презентации (например, с использованием слайдов

PowerPoint) запрещены. Цикл продолжается с обещанием по планированию спринта, на котором выбираются функции для следующего спринта.

2.4 Разработка на основе функций (FDD)

Авторы метода Feature Driven Development (FDD) [16, 37] Питер Кoad и Джеффе Луках характеризуют методологию как имеющую «досконально процесс, чтобы гарантировать масштабируемость и повторимость, а также поощрять творчество и инновации на всем протяжении процесса». [27]

В FDD постоянно подчеркивается важность наличия квалифицированных специалистов и опытных экспертов в предметной области. FDD построен на восьми передовых практиках: моделирование объектов предметной области; разработка по функциям; индивидуальное владение классами; команды разработчиков функций; инспекции; регулярные сборки; управление конфигурацией; отчетность/прозрачность результатов. Модели UML [23] широко используются в FDD.

2.4.1 Документы и артефакты

Описание функций, состоящее из набора функций, где функция является небольшими, полезными с точки зрения клиента, результаты; функция, имеющая ценность для клиента, которая может быть реализована за две недели или меньше. Если реализация функции займет более двух недель, ее необходимо дополнительно

разложить. Пакеты проектирования состоят из диаграмм последовательностей и диаграмм классов, а также информации о

дизайне методов. Отслеживание функций, диаграмма, которая перечисляет функции, которые должны быть реализованы, и даты завершения каждого этапа.

Диаграмма «Burn Up», диаграмма, на которой по оси X указаны даты (время). По оси Y отображается увеличивающееся количество завершенных функций. По мере завершения функций эта диаграмма показывает положительный наклон с течением времени.

2.4.2 Роли

о Менеджер проекта, являясь административным руководителем проекта, ответственным за отчетность о работе, управление бюджетами, борьбу за ресурсы, включая людей, оборудование и пространство, и управление ими. о Главный архитектор, отвечает за общий дизайн системы, включая проведение сессий по дизайну с командой. о Менеджер по разработке, отвечает за руководство повседневной деятельностью по разработке, включая разрешение конфликтов ресурсов. о Главный программист, как указано в идеях Брукса о хирургических бригадах [15], являясь опытным разработчиком, который выступает в качестве руководителя группы, наставника и разработчика для группы из трех-шести разработчиков. Главный программист предоставляет группе по разработке функций все сторонние знания о скелетной модели, участвует в высокоуровневом анализе требований и проектировании, а также помогает группе в низкоуровневом анализе, проектировании и разработке новых функций. о Владелец класса, отвечает за проектирование, кодирование, тестирование и документирование новых функций в классах, которыми он или она владеет. о Эксперты в предметной области, пользователи, клиенты, спонсоры, бизнес-аналитики и т. д., которые имеют глубокие знания о бизнесе, для которого разрабатывается продукт. о Команды разработки функций — это временные группы разработчиков, сформированные вокруг классов, в которых будут реализованы функции. Команда разработки функций динамически формируется для реализации функции и распускается после её реализации (через две недели или менее).

2.4.3 Процесс

Процесс FDD состоит из пяти инкрементных итеративных процессов, как показано на рисунке 5. Приводятся рекомендации по количеству времени, которое следует затрачивать на каждый из этих этапов, ограничивая время, затрачиваемое на общее планирование и архитектуру, и подчеркивая время, затрачиваемое на проектирование и реализацию функций. Процессы 1–3 выполняются в начале проекта и затем обновляются на протяжении всего цикла разработки. Процессы 4 и 5 выполняются поэтапно, циклами по две недели. Каждый из этих процессов имеет определенные критерии входа и выхода, причём критерий выхода процесса N является критерием входа процесса N-1.

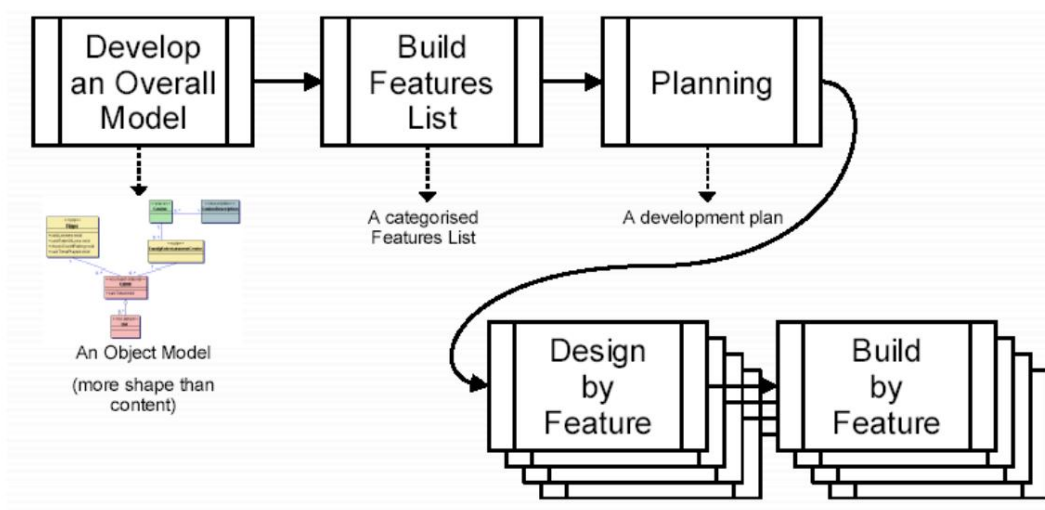


Рисунок 5: Обзор разработки на основе функций

оПроц ес с 1: Разработка общей модели (время : 10 проц ентов на начальном этапе, 4 проц ента в проц ес с е)

Члены команды, работающей над предметной областью и команды разработки совместно работают над пониманием области применения системы и её контекста. Для каждой области предметной области разрабатываются высокоуровневые объектные модели/диаграммы классов. В примечаниях к модели фиксируется информация о её форме и причинах выбора одних альтернатив и отклонения других.

оПроц ес с

2: Составление списка функций (время : 4% на начальном этапе, 1% в проц ес с е)

Полный список всех функций в проекте; функциональная декомпозиция, которая разбивает «бизнес-деятельность», запрошенную заказчиком, на функции, которые необходимо реализовать в программном обеспечении.

оПроц ес с 3: Планирование по

функциям (время : 2 проц ента изначально, 2 проц ента в проц ес с е)

Группа планирования, состоящая из менеджера проекта, менеджера по разработке и главного программиста, определяет порядок разработки функций. Планирование основано на зависимостях, рисках, сложностях, балансировке рабочей нагрузки, контрольных точках, требуемых клиентом. Бизнес-задачи имеют даты завершения помесячно/кварталами. Каждый класс закреплен за конкретным разработчиком. Функции объединяются в группы по техническим, а не по бизнес-причинам.

оПроц ес с 4: Проектирование по функциям (34% времени непрерывно в двух недельных итерациях).

Главный программист руководит разработкой пакетов проектирования и дорабатывает объектные модели с помощью атрибутов. Диаграммы последовательности часто создаются в рамках групповой работы.

Диаграммы классов и объектные модели разрабатываются владельцами классов.

Эксперты предметной области взаимодействуют с командой для уточнения требований к функциям.

Проекты

проверяются. оПроц ес с 5: Разработка по функциям (время : 43% времени в двух недельных итерациях)

Группа разработчиков реализует классы и методы, предусмотренные проектом.

Этот код проверяется и тестируется. Код переходит в сборку.

Прогресс отслеживается и отображается на этапах «Проектирование по функциям» и «Сборка по функциям».

Каждая функция имеет шесть контрольных точек: три из этапа «Проектирование по функциям» (просмотр предметной области, проектирование и проверка проекта) и три из этапа «Сборка по функциям» (код, проверка кода, переход к сборке). По завершении этих контрольных точек дается указание на диаграмме «Отслеживание по функциям», которая доступна для команды.

Когда функция достигает всех шести этапов, это отражается на диаграмме «Burn Up». Срок завершения всех функций, включая все шесть этапов, составляет максимум две недели.

3 Резюме Вэтой

в главе мы представили обзор гибкой модели разработки программного обеспечения и их характеристики проектов, которые могут подходить для использования этой модели.

Кроме того, мы представили обзоры трёх репрезентативных методологий: XP, Crystal и FDD. Обзор отличительных особенностей этих методологий представлен в таблице 1.

Гибкая методология	Отличительный фактор
Экстремальный Прогрессивное	- Предназначено для 10–12 совместно работающих объектно-ориентированных программистов - Четыре ценности: 12 четко определенных, дисциплинированных методов разработки - Минимальная архивная документация - Быстрые циклы обратной связи с клиентами
Кристалл	- и разработчиками - Настраиваемое семейство методологий разработки для небольших и очень большие команды - Методология зависит от размера команды и критичности проекта - Акцент на личном общении - Учитывайте людей, взаимодействия, сообщество, навыки, таланты и коммуникация как эффекты первого порядка - Начните с минимального процесса и наращивайте его по мере необходимости
Скрам	Оболочка управления проектами вокруг методологии, в которой определены практики разработчиков. 30-дневные спринты, в которых приоритеты не меняются - Ежедневные встречи Scrum-команды Scrum - Диаграмма сгорания для отображения прогресса - Масштабируемость для
Ориентированный на обязанности Разработка	- более крупных команд - Узкоспециализированные практики разработки - Пять подпроцессов, каждый из которых определяется критериями входа и выхода - Разработка представляет собой архитектурную структуру, объектные модели и диаграммы последовательности (модели UML используются повсеместно) - Двухнедельные материалы

Таблица 1: Краткое описание трех гибких методологий разработки и прогрессивного обеспечения

В этой главе представлено несколько ключевых идей гибких методологий. Ключи к успешному выбору и применению гибких методологий обобщены в таблице 2.

	Гибкие методы разработки (Agile) являются подмножеством итеративных и эволюционных методов. Итерации короткие, что позволяет обеспечить более своевременную обратную связь с командой проекта.
	В Agile Manifesto изложены приоритеты, лежащие в основе принципов и практик гибких методологий разработки и прогрессивного обеспечения.
	Экстремальное прогрессивное обеспечение основано на четырех ценностях и 12 конкретных практиках разработки и прогрессивного обеспечения.
	Семейство методологий Crystal настраивается в зависимости от особенностей проекта и команды.
	Методология Scrum в основном затрагивает принципы управления проектами. Она предоставляет команде свободу выбора конкретных практик разработки.
	Из четырех методологий, представленных в главе, FDD имеет наиболее тщательные методы анализа и проектирования.

Таблица 2: Ключевые идеи гибких методологий разработки и прогрессивного обеспечения

Существуют и другие гибкие методологии разработки и прогрессивного обеспечения, включая адаптивную разработку и прогрессивное обеспечение (ASD) [26], гибкое моделирование [2], метод разработки динамических систем (DSDM) [42], бережливую разработку [39] и Scrum [27, 41].

Кроме того, команды могут насстраивать гибкую методологию RUP. Все гибкие

Методологи или рас с матриваю разработку прог раммног ообес печения как эмпирический проц есс , требующий коротких ц иклов обратной с вя зи «проверка и адаптац ия » на протя жении вс ег о проекта.

Ссылки

- [1] ADM Inc., Что такое SCRUM: <http://controlchaos.com>. 2004. [2] [3] _____
- С. В. Эмблер, Г ибкое моделирование. Ньюй орк: John Wiley and Sons, 2002.
- М. Аоя ма, «Г ибкий проц есс разработки прог раммног ообес печения и ег о опыт», Между народная конференц ия по прог раммной инженерии, Киото, Япония , 1998 г ., с тр. 3-12.
- [4] К. Ауэр и Р. Миллер, XP Applied. Рединг , Мас с ачус етс : Addison Wesley, 2001.
- [5] В. Р. Базили и А. Дж. Тернер, «Итеративное улучшение: практический метод разработки прог раммног ообес печения », Труды IEEE по прог раммной инженерии, т. 1, № 4, с тр. 266–270, 1975.
- [6] К. Бек, «Экстремальное прог раммирование: приветс твуйте перемены». Рединг , Мас с ачус етс : Addison-Wesley, 2000.
- [7] К. Бек, Разработка через тестирование – на примерах . Бостон: Addison Wesley, 2003.
- [8] К. Бек, «Экстремальное прог раммирование: адаптац ия к переменам», второе издание. Рединг , Мас с ачус етс : Addison-Wesley, 2005.
- [9] К. Бек, М. Бидл, А. ван Беннекум, А. Кокберн, У. Каннинг ем, М. Фаулер, Дж. Греннинг , Дж. Х айс мит, А. Х ант, Р. Джеффрис , Дж. Керн, Б. Марик, Р. К. Мартин, С. Меллор, К. Цвабер, Дж. Сазерленди Д. Томас , «Манифест Agile», <http://www.agileAlliance.org>, 2001, 2001.
- [10] Д. Беллин и С.С. Симон, «Книжка-карточка CRC». Рединг , Мас с ачус етс : Addison-Wesley, 1997.
- [11] Б. Бём, «Г отовьтесь к г ибким методам с ос торожностью», IEEE Computer, т. 35, № 1, с тр. 64–69, 2002.
- [12] Б. Бём, «Спиральная модель разработки и с овершенс твования прог раммног ообес печения », Computer, т. 21, № 5, с тр. 61-72, май 1988 г .
- [13] Б. Бём и Р. Тернер, «Использование рис ка для баланс а между г ибкими и плановыми методами», IEEE Computer, т. 36, № 6, с тр. 57–66, июнь 2003 г .
- [14] Б. В. Бём, Экономика прог раммной инженерии. Энг лвуд Клиффс , НьюДжерси: Prentice-Hall, Inc., 1981.
- [15] Ф. П. Брукс , Мифический человеко-мес я ц , юбилейное издание: Addison-Wesley Publishing Company, 1995.
- [16] П. Коад, Э. Лефеври и Дж. ДеЛука, Моделирование Java в ц вете с ис пользованием UML: Prentice Hall, 1999.
- [17] А. Кокберн, «Г ибкая разработка прог раммног ообес печения ». Рединг , Мас с ачус етс : Addison Wesley Longman, 2001.
- [18] Б. Кертис , «Три проблемы, решаемые с помощью поведенческих моделей проц есс а разработки прог раммног ообес печения (доклад)», Между народная конференц ия по прог раммной инженерии, Питтс бург , Пенс ильвания , 1989, с тр. 398-399.
- [19] Т. ДеМарко, Slack: преодоление выг ораия , рутинной работы и мифа о полной эффективнос ти: Бродвей, 2002.
- [20] К. Эль Эмам, «Как добиться у спех а в небольших прог раммных проектах », Agile Project Management, т. 4, № 11, 2003 г .
- [21] SE Elmaghraby, EI Baxter и MA Vouk, «Подх од к моделированию и анализу проц есс ов производс тва прог раммног ообес печения », Intl. Trans. Operational Res, т. 2, № 1, с тр. 117-135, 1995. [22] _____
- Р. Фэрли, Конц епц ии прог раммной инженерии. Ньюй орк: McGraw-Hill, 1985.
- [23] М. Фаулер, UML Distilled. Рединг , Мас с ачус етс : Addison Wesley, 2000.
- [24] М. Фаулер, К. Бек, Дж. Брант, В. Опдайк и Д. Робертс , Рефакторинг : Улучшение дизайна сущес твующег о кода. Рединг , Мас с ачус етс : Addison Wesley, 1999.
- [25] М. Фаулер и Дж. Х айс мит, «Манифест Agile», в журнале «Разработка прог раммног ообес печения », авг уст 2001 г ., с тр. 28–32.

- [26] Дж. Хайсмит, Разработка адаптивного программного обеспечения . Нью-Йорк: Dorset House, 1999.
- [27] Дж. Хайсмит, Экспериментальная разработка программного обеспечения . Бостон, Массачусетс: Addison-Wesley, 2002.
- [28] WS Humphrey, Дисциплина программного инженерии. Reading, MA: Addison Wesley, 1995 г. од.
- [29] Р. Джеффрис, А. Андерсон и К. Хендриксон, «Экстремальное программирование». Аппер Сэддл Ривер, Нью-Джерси: Addison Wesley, 2001.
- [30] П. Кролл и П. Крухтен. «Рациональный унифицированный процесс: практическое руководство по RUP». Бостон: Addison Wesley, 2003.
- [31] П. Крухтен, «Рациональный унифицированный процесс: введение», третье издание. Бостон: Addison Wesley, 2004.
- [32] К. Ларман, «Гибкая и итеративная разработка: руководство для менеджеров». Бостон: Addison Wesley, 2004.
- [33] К. Ларман и В. Базили, «История итеративной и инкрементальной разработки», IEEE Computer, т. 36, № 6, с. тр. 47-56, июнь 2003 г.
- [34] Университет Лихай, «Гибкая конкуренция расширяется по всему миру», <http://www.ie.lehigh.edu/>, 1991, 1991.
- [35] Н. Нагппан, Л. Уильямс и М.А. Врук, «На пути к набору показателей для ранней оценки надежности программного обеспечения », Международный симпозиум по проектированию надежности программного обеспечения , краткий обзор, Денвер, штат Колорадо, 2003 г., с. тр.
- [36] Б. А. Огуннайк и У. Х. Рэй, Динамика процессов, моделирование и управление. Нью-Йорк: Oxford University Press, 1994.
- [37] С.Р. Палмер и Дж.М. Феллинг, «Практическое руководство по разработке на основе функций». Аппер Сэддл Ривер, Нью-Джерси: Prentice Hall PTR, 2002.
- [38] М.К. Полк, Б. Кертис и М.Б. Крисис, «Модель зрелости возможностей для программного обеспечения версии 1.1», Институт программного инженерии CMU/SEI-93-TR, 24 февраля 1993 г.
- [39] М. Поппендик и Т. Поппендик, Бережливая разработка программного обеспечения . Бостон: Addison Wesley, 2003.
- [40] Т. Поток и М. Врук, «Влияние бизнес-модели на производительность разработки объектно-ориентированного программного обеспечения », IBM Systems Journal, т. 36, № 1, с. тр. 140-161, 1997.
- [41] К. Шабер и М. Бидл, «Гибкая разработка программного обеспечения с использованием SCRUM». Аппер Сэддл Ривер, Нью-Джерси: Prentice-Hall, 2002.
- [42] Дж. Стэплтон, DSDM: Метод на практике, Второе издание: Addison Wesley Longman, 2003.
- [43] М. Врук и А.Т. Риверс, «Создание надежного программного обеспечения в условиях ограниченных ресурсов» Окружающая среда» в тематических исследованиях по надежности и обслуживанию WR Blischke и DN П. Мурти, редактор. Хобокен, Нью-Джерси: Wiley-Interscience, John Wiley and Sons, 2003, с. тр. 205. 231.
- [44] Л. Уильямс, «Программист XP: программист на несколько минут», IEEE Software, т. 20, № 3, с. тр. 16-20, май/июнь 2003 г.
- [45] Л. Уильямс и А. Кокберн, «Специальный выпуск по гибким методам», IEEE Computer, т. 36, № 3, июнь 2003 г.
- [46] Л. Уильямс и Р. Кесслер, «Гарное программирование: иллюстрированное». Реддинг, Массачусетс: Addison Wesley, 2003.